

CRICKET TACTICAL FIELD INTELLIGENCE SYSTEM

Next-Generation Data-Driven Batter × Bowler × Fielder × Situation Field Optimization Platform

MASTER IMPLEMENTATION PLAN

Objective: Build a professional-grade cricket tactical intelligence platform capable of dynamically recommending an optimal field configuration from the exact batter, bowler, fielders, match state, delivery context and historical data.

Development environment: Antigravity + Cursor + Obsidian

Initial build: Real-data-ready architecture using synthetic/sample data until the real dataset is supplied.

This document upgrades the supplied 14-page implementation plan into a data-driven, context-aware, multi-objective, simulation-backed, explainable and secure tactical decision-support platform.

1. Core Objective

Build a professional-grade cricket tactical intelligence platform capable of recommending an optimal field configuration dynamically.

The system should answer:

“Given everything happening right now, where should all 9 fielders be positioned to maximize the probability of the desired outcome?”

The old implementation technically considered batter + bowler + fielder, but its matchup logic was substantially rule-driven. The original plan explicitly described tactical reasoning that generated positions from batter risk and bowler conditions, followed by a run-saving optimizer. [filecite]turn0file0[L89-L105]

The new system must move from fixed tactical rules toward a data-driven, context-aware and optimization-backed decision engine.

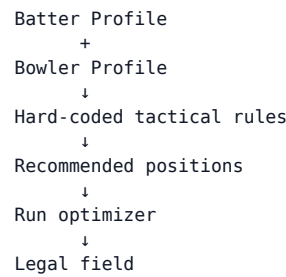
It must analyze the following simultaneously:

- Batter: individual scoring zones, boundary zones, dot-ball zones, dismissal patterns, pace vs spin performance, left/right-arm matchup, line and length vulnerability, shot selection, strike rotation, recent form, current innings state, balls faced and historical bowler matchups.
- Bowler: bowling type, arm angle, pace, swing, seam, spin, line, length, release position, bowling angle, variation usage, wicket-taking patterns, run-concession patterns, historical performance against this batter, current spell, current over, previous deliveries and planned delivery sequence.
- Fielder: catching ability, reaction time, jump/reach, throwing accuracy, throwing speed, ground fielding, boundary fielding, close-catching ability, position-specific performance, arm side, preferred positions and historical success at positions.
- Match situation: format, innings, over, ball, score, wickets, required run rate, current run rate, batter on strike, non-striker, partnership, powerplay/middle/death phase, new ball/old ball, target/chase situation and match importance.

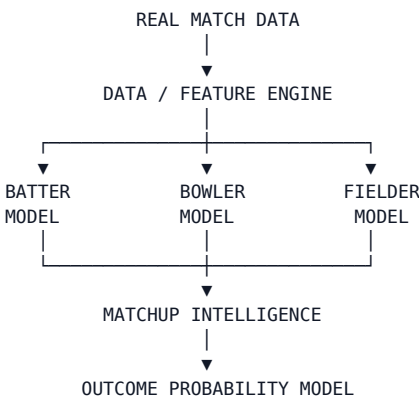
The original plan's input layer was narrower: Batter Profile, Bowler Profile, Fielder Pool and Match Phase. The new architecture makes full match state a first-class input. [filecite]turn0file0[L8-L24]

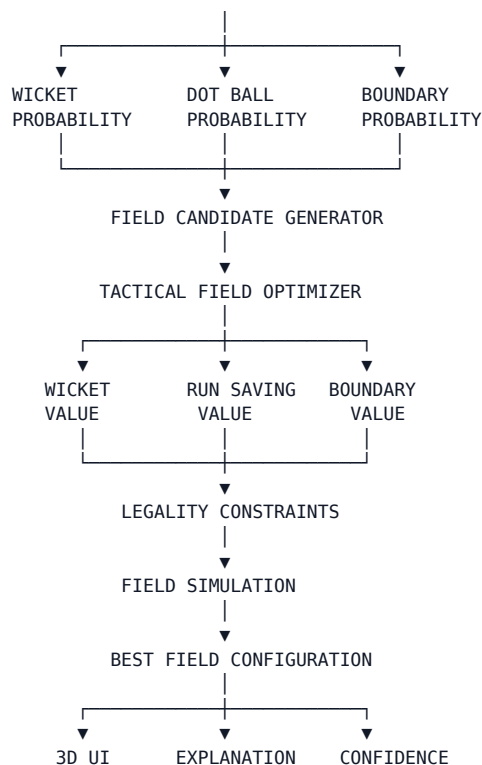
2. Fundamental Architectural Change

OLD SYSTEM



NEW SYSTEM





The key architectural principle is that the field itself becomes the output of an optimization problem, not a predefined answer.

3. Tactical Decision Engine 2.0

Create: `matchup_engine.py`

This replaces the old rule-heavy matchup engine. The old engine used explicit rules such as `edge_vs_pace HIGH + pace + outside_off → 1st Slip, 2nd Slip, Gully`. [\[filecite\]turn0file0\[L97-L124\]](#)

The new engine should calculate:

`P(Wicket | Batter, Bowler, Delivery, Field)`

`P(Dot | Batter, Bowler, Delivery, Field)`

`P(1/2/3 | Batter, Bowler, Delivery, Field)`

`P(Boundary | Batter, Bowler, Delivery, Field)`

`Expected Runs | Field`

`Expected Wicket Value | Field`

This is the fundamental upgrade: recommendations should be derived from estimated outcomes and tactical value rather than simply selecting positions from a fixed table.

4. Batter Intelligence Engine

Create: `batter_engine.py`

Instead of relying primarily on manually assigned values such as `edge_vs_pace`, `pull_mistime_vs_short_ball`, `sweep_risk_vs_spin`, etc., the system should eventually learn these values from historical deliveries.

Batter feature vector:

```

batter_id
handedness
career_stats
format_stats
phase_stats
pace_stats
spin_stats

```

```

left_arm_stats
right_arm_stats
line_stats
length_stats
shot_distribution
scoring_zones
dismissal_zones
boundary_zones
dot_ball_zones
recent_form
current_innings_state
balls_faced
runs_scored
strike_rate

```

5. Bowler Intelligence Engine

Create: bowler_engine.py

```

bowler_id
bowling_arm
bowling_type
pace
average_speed
maximum_speed
swing
seam
spin
release_angle
release_position
line_distribution
length_distribution
variation_distribution
wicket_patterns
boundary_concession_patterns
phase_performance
powerplay_performance
middle_over_performance
death_performance
batter_specific_history

```

The system should be capable of distinguishing two fast bowlers who create very different tactical opportunities against the same batter.

```

Bowler A
↓
outside-off + full
↓
Kohli edge probability ↑
↓
slip/gully value ↑

Bowler B
↓
hard length + body
↓
pull/miscue probability ↑
↓
deep square + fine leg value ↑

```

Therefore: same batter ≠ same field.

6. Batter × Bowler Matchup Engine

The matchup engine becomes the central intelligence layer.

```

      VIRAT KOHLI
        ×
    BOWLER PROFILE
        ↓

Outside Edge Risk    0.81
LBW Risk              0.23
Bowled Risk          0.17
Pull Miscue Risk     0.48
Drive Boundary Risk  0.71
Cover Boundary Risk  0.66
Straight Boundary Risk 0.41

```

Single Probability	0.52
Dot Probability	0.38
Boundary Probability	0.29

In the production version these values should be inferred from data/model outputs rather than manually entered numbers.

7. Delivery-Sequence Intelligence

Create sequence-aware intelligence so the system does not only ask what usually happens when a batter faces a bowler; it also asks what is likely to happen on the next ball given the previous deliveries.

```
Ball 1 → outswinger → dot
Ball 2 → outswinger → defended
Ball 3 → outswinger → single
Ball 4 → short → pull
Ball 5 → outswinger

NEXT BALL:
Expected delivery: Outside-off / full
Expected response: Drive / edge
Recommended field:
1st Slip
2nd Slip
Gully
Point
Cover
Mid-off
Fine Leg
Mid-on
Square Leg
```

This turns the system from a static field generator into a tactical sequence engine.

8. Fielder Intelligence 2.0

Create: fielder_engine.py

The old plan already included catching, arm, close-in skill, boundary skill, jump and preferred positions.

`filecite{turn0file0L58-L65}`

Upgrade every fielder profile with:

```
fielder_id
catching
reaction
jump
speed
acceleration
throw_accuracy
throw_speed
ground_fielding
boundary_fielding
close_catching
slip_skill
gully_skill
point_skill
cover_skill
square_leg_skill
fine_leg_skill
preferred_positions
historical_position_success
```

Then calculate a fielder-position value:

```
Fielder A → 1st Slip = 0.94
Fielder B → 1st Slip = 0.71
Fielder C → 1st Slip = 0.58
```

The system therefore decides not just that a slip is needed, but which available fielder maximizes expected wicket conversion at that position.

9. Field as an Optimization Problem

Instead of simply generating tactical positions and passing them to a greedy optimizer, the system should generate and score many legal candidate field configurations.

```
FIELD A
↓
Expected Wickets = 0.18
Expected Runs = 7.4
Boundary Risk = 0.21

FIELD B
↓
Expected Wickets = 0.24
Expected Runs = 7.8
Boundary Risk = 0.18

FIELD C
↓
Expected Wickets = 0.21
Expected Runs = 6.9
Boundary Risk = 0.24
```

The optimizer then chooses according to the current tactical objective.

10. Multi-Objective Field Optimizer

Create: field_optimizer.py

The objective should not simply maximize ERS.

```
FIELD_SCORE =

 $\alpha$  × Expected_Wicket_Value

+  $\beta$  × Expected_Dot_Value

+  $\gamma$  × Expected_Run_Saving

+  $\delta$  × Boundary_Prevention

+  $\epsilon$  × Fielder_Quality

+  $\zeta$  × Tactical_Value

-  $\lambda$  × Risk
```

The coefficients should initially be configurable and eventually be learned/calibrated from data.

11. Continuous Field Positioning

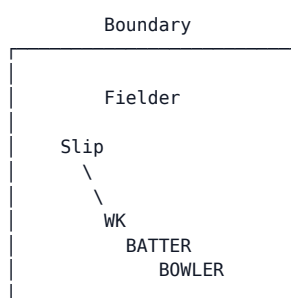
The original plan defines around 30 named positions with normalized coordinates and LHB/RHB mirroring. `filecite{turn0file0L71-L87}` Keep these as semantic positions, but introduce continuous positioning.

```
1st Slip:
x = 17.4–20.5m
y = 8.5–11.5m
```

The optimizer can then determine adjustments such as: “Move 1st Slip 1.4 m wider.” This is closer to real tactical analysis than selecting only a fixed named coordinate.

12. 3D Field Model

Create: field_geometry.py



The 3D UI should support rotate, zoom, pan, top-down view, tactical view, batter POV, bowler POV, fielding POV, animated ball trajectory, player movement and field changes between balls.

13. UI/UX Architecture

Do not use Streamlit as the final production UI. The old plan used Streamlit + Plotly for the dashboard, which is suitable for an early prototype but not ideal for the final 3D sports intelligence product.

filecite turn0file0L269-L299

Layer	Recommended technology
Frontend	Next.js, React, TypeScript, Tailwind CSS
3D	React Three Fiber, Three.js
Animation	Framer Motion
Backend	Python, FastAPI
AI / ML	scikit-learn, XGBoost / LightGBM, PyTorch when required
Optimization	OR-Tools, MILP / CP-SAT
Database	PostgreSQL
Cache	Redis
Data processing	Pandas, Polars, NumPy
Testing	Pytest, Playwright
Deployment	Docker

14. Main 3D Screen

INDIA TACTICAL INTELLIGENCE

IND vs AUS
2nd ODI
IND 184/3
34.2 overs

CENTER:
Interactive 3D cricket field
Players as tactical markers
Field labels
30-yard circle
Boundary
Ball trajectory
Heatmaps / overlays

15. Left Panel — Match Context

MATCH
Format ODI
Innings 2
Over 34.2
Score 184/3
Target 287

BATTER
Virat Kohli
RHB
Runs: 73
Balls: 81

BOWLER
Right-arm Fast
Overs: 7.2
Wickets: 1
Economy: 4.21

16. Right Panel — Tactical Intelligence

TACTICAL RECOMMENDATION

PRIMARY PLAN
Outside-Off Attack

Confidence
 82%

Expected Wicket
12.8%

Expected Boundary
9.4%

Expected Runs
5.91

- WHY?
- Batter has elevated edge probability
 - Bowler's outside-off delivery is highly effective
 - Gully covers secondary edge trajectory
 - 1st slip has highest catch conversion
 - Deep square remains necessary against pull

17. Field Change Animation

CURRENT

- Slip
- Gully
- Point
- Cover
- Mid-off

SIMULATED CHANGE

Slip → wider
Gully → deeper
Point → backward
Fine Leg → square

Animate players physically moving so that the UI communicates tactical change rather than merely displaying a new static field.

18. Tactical Explanation Engine

Create: explanation_engine.py

1st Slip selected

Reason:
Kohli's historical outside-edge probability against this bowler profile is elevated.

Expected edge opportunity: 18.7%

Fielder selected:
Fielder X

Reason:
Highest estimated catch conversion among available fielders at this position.

19. Confidence System

Recommendation Confidence

82%

Data confidence	91%
Matchup confidence	84%
Fielder confidence	88%

Model confidence 79%
Overall confidence 82%

△ LOW DATA CONFIDENCE

Only 14 comparable deliveries available.

Recommendation based partly on
archetype-level evidence.

20. Real Data Architecture

```
RAW DATA
↓
VALIDATION
↓
NORMALIZATION
↓
DATA CLEANING
↓
DELIVERY STANDARDIZATION
↓
FEATURE ENGINEERING
↓
FEATURE STORE
↓
TRAINING DATASET
↓
MODEL
↓
MODEL VALIDATION
↓
MODEL REGISTRY
↓
PRODUCTION
```

Do not feed raw CSV directly into the model.

21. Proposed Delivery-Level Data Schema

```
match_id
innings
over
ball
batter_id
non_striker_id
bowler_id
batter_hand
bowler_hand
bowler_type
delivery_speed
line
length
swing
seam
spin
shot_type
shot_direction
runs
extras
boundary
six
four
wicket
dismissal_type
fielder_involved
field_position_snapshot
```

If the future dataset does not contain a field, the architecture should mark it as unavailable rather than inventing it.

22. Feature Engineering

Create: feature_engine.py

```
batter_vs_pace_SR
```

```

batter_vs_spin_SR
batter_vs_bowler_type
batter_edge_rate
batter_pull_rate
batter_drive_rate
batter_boundary_probability
bowler_wicket_rate
bowler_dot_rate
bowler_boundary_rate
bowler_line_distribution
bowler_length_distribution
fielder_position_success
fielder_catch_probability
batter × bowler × delivery_type

```

The batter × bowler × delivery_type interaction is especially important because tactical behavior depends on the combination, not isolated player statistics.

23. Machine Learning Layer

Do not begin with a giant neural network. Start with strong tabular models and establish trustworthy baselines.

Model	Purpose
Expected Runs Model	Predict 0 / 1 / 2 / 3 / 4 / 6 / wicket outcome distribution
Wicket Probability Model	Estimate dismissal probability
Boundary Probability Model	Estimate four/six or boundary-event probability
Shot/Zone Prediction Model	Predict likely shot and directional zone
Fielder Success Model	Estimate conversion/fielding effectiveness by position
Sequence Model (later)	Model consecutive-delivery tactical evolution

24. Simulation Engine

Create: simulation_engine.py

```
simulate 1,000+ possible outcomes
```

```

FIELD A
Wicket:      16.8%
Dot:         34.1%
Boundary:    8.2%
Expected Runs: 0.79

```

```

FIELD B
Wicket:      19.4%
Dot:         31.7%
Boundary:    7.1%
Expected Runs: 0.74

```

Simulation allows the optimizer to compare candidate fields using expected outcomes rather than relying on a single deterministic score.

25. Situational Objective

The best field depends on the tactical objective.

Situation	Objective shift
Protect a 20-run lead	Run prevention ↑, boundary protection ↑, risk tolerance ↓
Need a wicket immediately	Wicket probability ↑↑, close catchers ↑, run-saving priority ↓
Batter needs 12 from 6	Boundary zones ↑↑, six-hitting zones ↑
Batter on 99	Single prevention + psychological/tactical context + historical scoring patterns

26. Match State Engine

Create: match_state.py

```
score
wickets
over
ball
target
required_rate
current_rate
partnership
batter_runs
batter_balls
non_striker
bowler_spell
field_restrictions
```

TACTICAL OBJECTIVE

WICKET > RUN PREVENTION

Reason:

Required RR is manageable.

Primary strategic value is breaking partnership.

27. Rules Engine 2.0

The original rules engine already validates player count, wicketkeeper, bowler, fielding restrictions, duplicate/overlap concerns and format phase constraints. [filecite\turn0file0\L174-L199](#)

```
rules_engine/
├── odi_rules.py
├── t20_rules.py
├── test_rules.py
├── field_geometry.py
└── validator.py
```

Validate player count, keeper, bowler, fielding restrictions, position legality, distance constraints, duplicate positions, boundary restrictions and format-specific rules.

Rules should be configurable/versioned rather than permanently hard-coded because cricket regulations can change.

28. Security Architecture

Because the long-term objective is real professional-team data, security must be designed from day one.

Authentication:
RBAC

Roles:
ADMIN
COACH
ANALYST
SCOUT
VIEWER

Security:
HTTPS/TLS
JWT/session security
encrypted database
encrypted backups
environment secrets
API rate limiting
input validation
file validation
audit logging
dependency scanning
Docker isolation

Never store API keys, passwords, database credentials or cloud credentials inside Git.

29. Data Security

```
RAW DATA
PROCESSED DATA
MODEL DATA

/data/raw
/data/processed
/data/features
/models

dataset_id
version
source
upload_time
checksum
schema_version
```

30. Model Versioning

```
model_id
version
training_dataset
training_date
features
metrics
validation_score
```

```
BATTER_MODEL
v0.1.0
```

```
Dataset:
India Domestic + International
```

```
Training:
2026-08-22
```

```
Validation:
0.81
```

Never silently replace a deployed model.

31. Human-in-the-Loop

The system should recommend, not blindly command.

```
AI RECOMMENDATION
```

```
[ ACCEPT ]
```

```
[ MODIFY ]
```

```
[ REJECT ]
```

```
COACH:
Move Gully 3m deeper
```

```
SYSTEM:
Recalculating...
```

```
Expected wicket:
18.2% → 17.6%
```

```
Boundary prevention:
91% → 94%
```

```
Recommendation:
KEEP MODIFIED FIELD
```

This makes the product an AI tactical assistant rather than an unsafe black-box decision maker.

32. Obsidian Role

Use Obsidian as the project's knowledge and architecture brain.

```
Cricket Intelligence/
|
|— 00_Project/
|   |— Vision
|   |— Requirements
|
```

```

└─ Roadmap

01_Architecture/
├─ System Architecture
├─ Data Architecture
├─ ML Architecture
└─ Security Architecture

02_Cricket/
├─ Field Positions
├─ Matchups
├─ Bowling Tactics
└─ Fielding Tactics

03_Data/
├─ Dataset Schema
├─ Features
└─ Data Dictionary

04_ML/
├─ Models
├─ Experiments
└─ Evaluation

05_UI/
├─ Screens
├─ Components
└─ UX

06_Security/
07_Testing/
08_Decisions/

```

Use Obsidian to document why architectural and tactical decisions were made.

33. Cursor Role

Use Cursor as the primary engineering environment for Python backend, ML, database, API, tests, frontend integration, refactoring and debugging.

Do not instruct Cursor to “build the whole application” in one shot. Build module-by-module.

34. Antigravity Role

Use Antigravity primarily for the product/UI/visual development workflow: 3D cricket field, player models, animations, dashboard, tactical overlays, heatmaps, charts, match timeline and field transitions.

The visual layer should communicate the intelligence rather than merely decorate it.

35. Project Architecture

```

cricket-intelligence/
├─ apps/
│   ├── web/
│   │   ├── app/
│   │   ├── components/
│   │   ├── three/
│   │   └─ dashboard/
│   └─ api/
├─ backend/
│   ├── api/
│   ├── domain/
│   ├── engines/
│   ├── optimization/
│   ├── simulation/
│   └─ services/
├─ ml/
│   ├── features/
│   ├── models/
│   ├── training/
│   └─ evaluation/

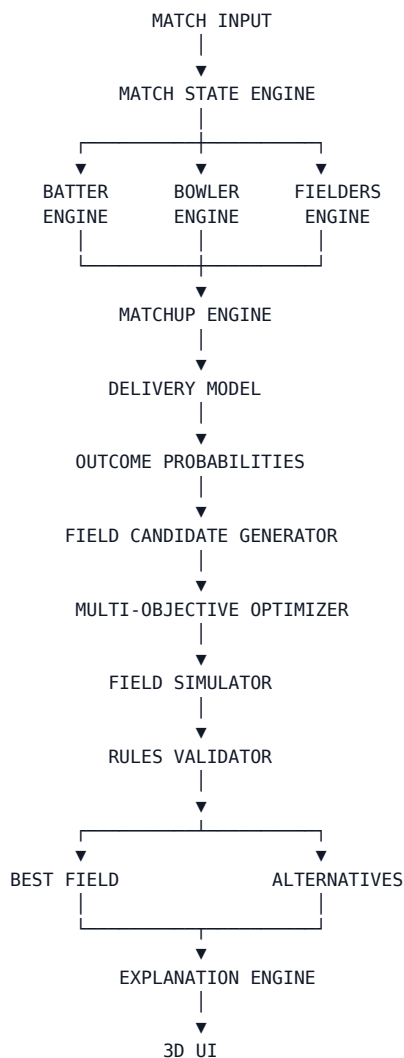
```

```

├── registry/
├── data/
│   ├── raw/
│   ├── processed/
│   ├── features/
│   └── schemas/
├── rules/
│   ├── odi/
│   ├── t20/
│   └── test/
├── security/
├── tests/
├── docs/
├── notebooks/
├── configs/
├── docker/
├── .env.example
├── README.md
└── pyproject.toml

```

36. Master Pipeline



37. Three Field Recommendations

Do not show only one field. Provide three strategic options.

Field	Wicket	Expected Runs	Boundary Risk
Aggressive / Wicket Field	18.7%	0.91	11.2%

Field	Wicket	Expected Runs	Boundary Risk
Balanced Field	15.4%	0.78	8.1%
Run-Prevention Field	11.2%	0.64	5.4%

The coach can choose based on strategy.

38. “Why Did the Field Change?”

```
FIELD CHANGE DETECTED

Previous:
1st Slip
2nd Slip
Gully

New:
1st Slip
Gully
Backward Point

WHY?

• Batter's scoring probability through cover increased
• Bowler shifted from full → hard length
• Edge probability decreased
• Cover-region boundary probability increased
• Match state now favors run prevention
```

39. Testing Strategy

The old plan already included tests for profiles, matchup logic, field optimization, rules, metrics and the full optimizer. `filecite\turn0file0\L300-L329` Expand that test architecture.

Test layer	Coverage
Unit tests	profiles, features, matchup, fielder, optimizer, simulation, rules, metrics
Integration tests	data → model → optimizer → UI
Security tests	authentication, authorization, malicious upload, invalid dataset, SQL injection, API abuse, secret exposure
UI tests	Playwright: field rendering, dragging, rotation, zoom, field changes, filters, mobile/desktop

40. Golden Tactical Scenario Tests

```
SCENARIO-001

Batter:
Kohli

Bowler:
Right-arm fast

Phase:
Powerplay

Ball:
Outside-off

Expected tactical characteristics:
Slip
Gully
Point
Cover
...
```

The model output should be evaluated against tactical properties and outcome quality rather than being forced to reproduce one exact historical field, because multiple fields can be strategically valid.

41. Performance Targets

Operation	Target
Data lookup	<100 ms
Prediction	<200 ms
Field optimization	<1 sec target
3D rendering	60 FPS target
Full recommendation	<2 sec target

If optimization takes longer, use a two-stage experience: fast recommendation followed by detailed optimization and a high-confidence recommendation.

42. Development Phases

Phase	Build
0 — Architecture	Project structure, documentation, schemas, API contracts, database schema, security baseline
1 — 3D Tactical UI Prototype	3D ground, players, positions, drag/drop, camera, field labels, match panel
2 — Deterministic Intelligence	Batter, bowler, fielder, match-state, rules and optimizer using sample data
3 — Statistical Intelligence	Historical probabilities, matchup statistics, zone models, fielder performance
4 — ML	Expected runs, wicket, boundary, shot and fielder models
5 — Simulation	Candidate fields, Monte Carlo simulation, expected outcomes and alternatives
6 — Real Dataset Integration	Upload, schema validation, cleaning, features, training, evaluation, calibration, deployment
7 — Professional Tactical UI	3D animations, heatmaps, ball trajectories, field transitions, timeline, confidence, explanations
8 — Security Hardening	RBAC, encryption, audit logs, secure API, dataset isolation, model versioning, testing, backup/recovery

43. Version Roadmap

V0.1 → 3D field, manual players, manual batter/bowler, basic rules
V0.2 → Batter intelligence, Bowler intelligence, Fielder intelligence
V0.3 → Matchup engine
V0.4 → Optimization engine
V0.5 → Simulation
V0.6 → Statistical models
V0.7 → ML models
V0.8 → Real dataset integration
V0.9 → Professional 3D tactical dashboard
V1.0 → Secure, validated, explainable, real-data-ready tactical decision-support platform

44. First Milestone — Dynamic Tactical Field

The old milestone was essentially: Kohli + Right-arm Fast + Over 2 → at least one slip. The new milestone must be substantially more demanding. [filecite turn0file0 L330-L339](#)

Input:

Batter
Bowler
9 fielders
Format
Over
Score
Wickets
Current ball
Previous deliveries

The system must:

- Analyze batter.
- Analyze bowler.
- Analyze matchup.
- Analyze current match state.
- Analyze every fielder.
- Generate candidate fields.
- Optimize them.
- Check legality.
- Simulate outcomes.
- Select the top 3 fields.
- Assign individual fielders.
- Give confidence.
- Explain every important placement.
- Render the result in 3D.

Then allow a coach/analyst to modify the field and have the system immediately recalculate tactical consequences.

COACH:
Move Gully 3m deeper

SYSTEM:
Recalculating...

Expected wicket:
18.2% → 17.6%

Boundary prevention:
91% → 94%

Recommendation:
KEEP MODIFIED FIELD

45. Final Architectural Principle

Do not build a lookup table disguised as AI.

BAD:

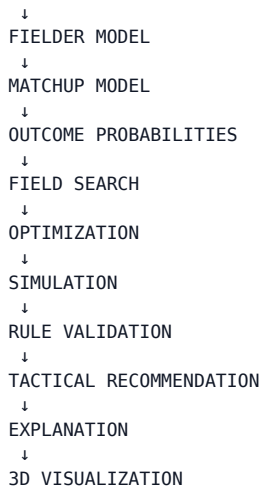
IF batter X
AND bowler Y
THEN put fielder Z at position A

Also avoid merely replacing the lookup table with a single threshold rule:

HIGH edge risk
→ Slip

GOOD:

DATA
↓
CONTEXT
↓
BATTER MODEL
↓
BOWLER MODEL



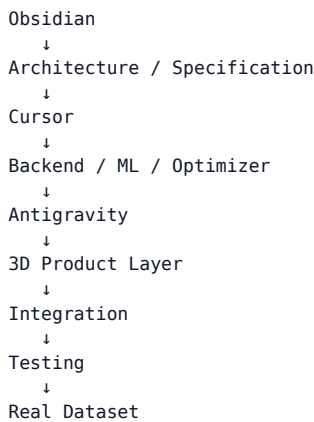
The field becomes the output of a multi-objective optimization process.

The engineering target can be “precise and rigorous,” but the system should not claim perfect prediction. Cricket is probabilistic. A professional-grade product should expose probabilities, confidence, uncertainty, alternatives, data coverage and human override rather than pretending the AI knows exactly what will happen.

The old implementation remains valuable as a foundation: its two-stage idea of reserving tactical wicket positions and then optimizing the remaining fielders is worth retaining, as are ERS/EWO/CDS, legality validation, LHB/RHB mirroring and the data-loader concept. [filecite\[turn0file0L149-L173\]](#)
[filecite\[turn0file0L201-L227\]](#)

The major upgrade is to turn those components into a data-driven, multi-objective, simulation-backed tactical intelligence platform rather than a collection of cricket rules.

Recommended build order:



This order prevents the UI from becoming beautiful while the underlying tactical engine is still fundamentally a hard-coded prototype.

Appendix A — Alignment With the Supplied Old Plan

The supplied 14-page implementation plan described a three-entity defensive decision engine: batter profile, bowler profile and fielder pool, with match phase, tactical reasoning, a tactical-position stage, a run-saving optimizer, rules/legality validation, metrics and an explanation layer. [filecite\turn0file0\L2-L24]

It also defined sample batter profiles, bowler archetypes and 11 fielder profiles; around 30 field positions with normalized coordinates and LHB/RHB mirroring; a matchup engine; a zone-based field optimizer; a rules engine; ERS/EWO/CDS metrics; a real-data loader; a hybrid orchestrator; a Streamlit dashboard; tests; and a staged build order. [filecite\turn0file0\L34-L87] [filecite\turn0file0\L149-L227] [filecite\turn0file0\L269-L329]

The upgraded plan keeps the useful foundations while changing the intelligence layer from mainly expert-rule selection into context-aware statistical/ML inference, field search, multi-objective optimization, simulation, continuous positioning, fielder-position matching, confidence, explainability, human-in-the-loop control and production security.

Appendix B — Implementation Principles

- Build deterministic baselines before ML so every model can be compared against a transparent reference.
- Keep cricket rules separate from tactical intelligence so regulations can be updated without rewriting the model.
- Separate raw, processed, feature and model data.
- Version datasets, features and models.
- Treat missing data explicitly; never fabricate missing cricket events.
- Use confidence and uncertainty throughout the product.
- Return multiple viable fields instead of pretending there is always one uniquely correct field.
- Let coaches/analysts override AI recommendations and quantify the impact of modifications.
- Use real data only after validation, cleaning, schema verification and model evaluation.
- Keep the final UI fast enough for live tactical workflows.

Appendix C — Definition of Success

A successful V1 is not simply a beautiful 3D cricket field. It is a system in which the 3D field is the visible output of a defensible analytical pipeline that can explain why each major fielder was selected, quantify the expected tactical value, obey the relevant fielding rules, compare alternatives and adapt when batter, bowler, fielder or match conditions change.